

Process Management

2025 Semester 1 SOFTENG 370: Operating Systems

Talia Xu

Lecture 5

2.0.0

Putting together: fork, wait and exec

On Linux, what happens when you run a command?

Let's try to run `sleep 2`

On Unix, the Kernel Launches A Single User Process

After the kernel initializes, it creates a single process from a program

This process is called `init`, and it looks for it in `/sbin/init`

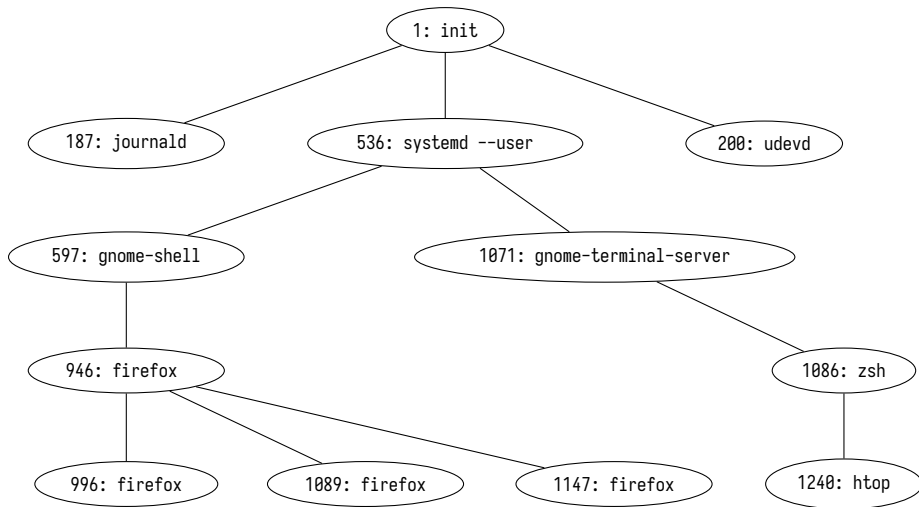
- Responsible for executing every other process on the machine

- Must always be active, if it exits the kernel thinks you're shutting down

For Linux, `init` will probably be `systemd` but there's other options

Aside: some operating systems create an "idle" process that the scheduler can run

A Typical Process Tree on the Virtual Machine



How You Can See Your Process Tree

Use htop

You can press F5 to switch between tree and list view

Processes Are Assigned a Process ID (pid) On Creation

The process ID is just a number, and is unique for every **active** process

On most Linux systems the maximum pid 32768, and 0 is reserved (invalid)

Eventually the kernel will recycle a pid, after the process dies, for a new process

Remember: each process has its own *address space* (independent view of memory)

Maintaining the Parent/Child Relationship

Previously, we made sure that our parent exited last (by using sleep)

What happens if the parent process exits first, and no longer exists?

The Parent Process is Responsible for Its Child

The operating system sets the exit status when a process terminates (the process terminates by calling `exit`)

It can't remove its PCB yet

The minimum acknowledgment the parent has to do is read the child's exit status

There's two situations:

1. The child exits first (zombie process)
2. The parent exits first (orphan process)

A Zombie Process Waits for Its Parent to Read Its Exit Status

The process is terminated, but it hasn't been acknowledged

A process may have an error in it, where it never reads the child's exit status

The operating system can interrupt the parent process to acknowledge the child

- It is just a suggestion and the parent is free to ignore it

- This is a basic form of IPC called a signal

The operating system has to keep a zombie process until it's acknowledged

- If the parent ignores it, the zombie process needs to wait to be re-parented

An Orphan Process Needs a New Parent

The child process lost its parent process

The child still needs a process to acknowledge its exit

The operating system re-parents the child process to init

The init process is now responsible to acknowledge the child

orphan-example.c The Parent Exits Before the Child

```
int main(int argc, char *argv[]) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("Child parent pid: %d\n", getppid());
        sleep(2);
        printf("Child parent pid (after sleep): %d\n", getppid());
    }
    else {
        sleep(1);
    }
    return 0;
}
```

zombie-example.c The Parent Monitors the Child To Check Its State

```
pid_t pid = fork();

if (pid == 0)
    sleep(2);

else {
    int ret;
    sleep(1);
    printf("Child process state: ");
    ret = print_state(pid);
    sleep(2);
    printf("Child process state: ");
    ret = print_state(pid);
}
```

You're Responsible for Managing Processes

The operating system maintains a strict parent/child relationship

You should be able to identify (and prevent) the following:

- Zombie processes
- Orphan processes

Review Question

What is the parent process of the second child after 1 second? 2 seconds?

```
int main() {  
    if (fork() == 0) {  
        if (fork() == 0)  
            sleep(2);  
        exit(0);  
    } else {  
        sleep(3);  
        exit(0);  
    }  
}
```

Review Question

What is the parent process of the second child after 1 second? 2 seconds?

```
int main() {  
    if (fork() == 0) {  
        if (fork() == 0)  
            sleep(2);  
        exit(0);  
    } else {  
        sleep(3);  
        exit(0);  
    }  
}
```

Review Question

For each program shown below, state whether it will produce the same output each time it is run

```
int main() {  
    int i = 4;  
    while (i != 0) {  
        int pid = fork();  
        if (pid == 0) {  
            i--;  
        } else {  
            printf("%d\n", i);  
            exit(0);  
        }  
    }  
    return 0;  
}
```

Review Question

The `waitpid()` system call suspends execution of the calling process until a child specified by `pid` argument has changed state.

```
int main() {  
    int i = 4;  
    while (i != 0) {  
        int pid = fork();  
        if (pid == 0) {  
            i--;  
        } else {  
            waitpid(pid, NULL, 0);  
            printf("%d\n", i);  
            exit(0);  
        }  
    }  
    return 0;  
}
```

Review Question: fork()

How many times does Hello get printed?

```
int main() {  
    fork();  
    fork();  
    fork();  
    printf("Hello\n");  
    return 0;  
}
```

Review Question: Process API

What is the output?

```
int x = 3;
void main()
{
    while(x > 0){
        int ret = fork();
        if(ret == 0){
            printf("x = %d\n", x);
            x--;
            execl("./a.out", "a.out", "", 0);
        } else
            wait(NULL);
    }
}
```